Sistema de Anotaciones Semánticas para IAs

Aunque no dispongamos de herramientas automatizadas complejas, podemos "programar" el comportamiento de la IA mediante un sistema de anotaciones basado en texto. Las IAs modernas entienden muy bien los metadatos y las instrucciones restrictivas si se les enseñan previamente.

1. El Archivo de Reglas (El "AGENTS.md")

Para que la IA respete estas anotaciones, debe existir un archivo en la raíz del proyecto (por ejemplo, `AGENTS.md` o `.cursorrules`) que la IA debe leer **al iniciar cualquier sesión**.

> **Instrucción para poner en el archivo de reglas de la IA:**
>
> *"IMPORTANTE: Antes de modificar cualquier archivo, debes leer y respetar obligatoriamente las anotaciones en la cabecera del mismo (marcadas con @). Si un archivo tiene un @limit(lines: N) y tus cambios lo sobrepasan, DEBES refactorizar el código en piezas más pequeñas antes de terminar tu respuesta."*

2. Las Anotaciones Principales

Todas estas anotaciones deben incluirse como comentarios en la parte superior de los archivos fuente.

A) La Cabecera Principal `@use`

Indica a la IA de qué elementos hace uso este archivo.

```
// @use(kind, contract, limit)
```

B) El Tipo de Archivo `@kind`

Clasifica el archivo de manera estricta para que la IA aplique los patrones correctos.

```
// @kind(component)
// Opciones: component, hook, service, store, util, page, config
```

C) El Contrato `@contract`

Obliga a la IA a definir exactamente qué entra, qué sale, y qué cambia.

```
// @contract(in: UserData -> out: boolean, sideEffect: Update DB)
```

D) El Límite Estricto `@limit`

El concepto más importante para evitar el "código espagueti". La IA debe negarse a sobrepasarlo.

```
// @limit(lines: 80)
```

3. El Loop de Aprendizaje `@learn-error`

Las IAs suelen tropezar más de una vez con el mismo error si el contexto de la conversación es largo. Para evitarlo, usamos la anotación `@learn-error` (generalmente almacenada en el archivo de reglas global o en un documento de historial de la IA).

Cuando la IA comete un error, pídele que documente la solución usando este formato:

```
// @learn-error(
//   id: 'ERR-001',
//   input: 'Se intentó mutar el estado directamente (state.push(item))',
//   expected: 'Retornar un nuevo arreglo ( [...state, item] )',
//   actual: 'La UI no se actualizó porque la referencia no cambió',
//   fix: 'Se usó el operador spread para crear inmutabilidad'
// )
```

¿Por qué funciona esto?
Cuando la IA lee esto al iniciar, carga esta lección en su contexto inmediato, evitando que en esa sesión vuelva a cometer exactamente el mismo error.

4. Ejemplo de un Archivo Completo

Así se vería la parte superior de un archivo siguiendo el sistema de anotaciones:

```
// @use(kind, contract, limit)
// @kind(hook)
// @contract(in: id de usuario -> out: perfil, sideEffect: fetch)
// @limit(lines: 60)

import { useState, useEffect } from 'react';

export function useUserProfile(userId) {
  // Lógica del código aquí...
}
```

5. El Ciclo de Vida del Trabajo con IA (El Workflow PCA)

Para que el desarrollo sea seguro y profesional, no basta con poner etiquetas. Debes seguir (y exigirle a tu IA que siga) un flujo de trabajo estructurado para CADA tarea que realicen. Este es el ciclo de vida obligatorio, basado en la metodología PCA, que debes interiorizar:

1. **CONTEXT (Contexto):** Asegúrate de que la IA ha leído tu `AGENTS.md` y tiene claro el estado actual del proyecto.
2. **ANALYZE (Análisis):** Evalúa con la IA qué piezas del sistema están involucradas.
3. **PLAN (Planificación): Paso Crítico.** Nunca dejes que la IA empiece a escribir código directamente. Exígele un plan escrito (El "Cómo" del PCA) y apruébalo.
4. **RESEARCH (Investigación):** Pide a la IA que revise si en tu biblioteca personal hay algo que se pueda reutilizar.
5. **ANALYZE IMPACT (Impacto):** Exígele a la IA predecir qué otras partes del sistema podrían romperse (El "A qué" del PCA).
6. **MUTATE (Mutación):** Autoriza a la IA a escribir el código, forzándola a incluir las anotaciones.
7. **VALIDATE (Validación): Tu Responsabilidad.** Revisa el código, pruébalo y verifica visualmente que no violó las reglas ni límites de líneas.
8. **INTEGRATE (Integración):** Si todo compila, integra el código al repositorio principal.
9. **LEARN (Aprendizaje):** Documenta cualquier fallo de la IA usando `@learn-error` en tu historial.
10. **CLOSE (Cierre):** Da por terminada la tarea. Inicia un chat nuevo para el próximo Sprint, evitando así la Amnesia de Contexto.

Conclusión para el Estudiante

Este sistema no requiere instaladores. Al forzar a la IA a usar las anotaciones y respetar los 10 pasos del ciclo de vida, estás construyendo el escudo protector de los ingenieros modernos. ***Tú te conviertes en el Arquitecto que audita, y la IA en la herramienta que ensambla.***